Step 1: Read an integer from the user.
The program will take an integer as input.
Step 2: Use conditional statements to check the number's range.
An if-else if-else structure can be used to check the conditions.
If the number is less than 10, print "one digit".
If the number is greater than or equal to 10 but less than 100, print "two digit".
Otherwise, print "integer".
Step 3: Write the C code.
C

```c
#include <stdio.h>

int main() {
    int num;
    printf("Enter an integer: ");
    scanf("%d", &num);
    if (num < 10) {
        printf("one digit\n");
    } else if (num >= 10 && num < 100) {
        printf("two digit\n");
    } else {
        printf("integer\n");
    }
    return 0;
}
```

2. Check for a leap year
Step 1: Read a year from the user.
The program will take a year as an integer input.
Step 2: Apply the leap year logic.
A year is a leap year if it is divisible by 4. However, years divisible by 100 are not leap years, unless they are also divisible by 400. This can be expressed with the condition:
$$(year \% 4 == 0 \text{ && } year \% 100 != 0) \text{ || } (year \% 400 == 0)$$
Step 3: Write the C code.
C

```c
#include <stdio.h>

int main() {
    int year;
    printf("Enter a year: ");
    scanf("%d", &year);
    if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)) {
        printf("%d is a leap year.\n", year);
    } else {
        printf("%d is not a leap year.\n", year);
    }
    return 0;
}
```

3. Check for character type
Step 1: Read a character from the user.
The program will take a single character as input.
Step 2: Use conditional logic to check the character's type.
An alphabet is a character between 'a' and 'z' or 'A' and 'Z'.
A digit is a character between '0' and '9'.
If it's neither, it's a special character.
Step 3: Write the C code.
C

```
#include <stdio.h>

int main() {
    char ch;
    printf("Enter a character: ");
    scanf(" %c", &ch);
    if ((ch >= 'a' && ch <= 'z') || (ch >= 'A' && ch <= 'Z')) {
        printf("%c is an alphabet.\n", ch);
    } else if (ch >= '0' && ch <= '9') {
        printf("%c is a digit.\n", ch);
    } else {
        printf("%c is a special character.\n", ch);
    }
    return 0;
}
```

4. Check for vowel or consonant
Step 1: Read an alphabet from the user.
The program will take a single character as input. This problem assumes the input is an alphabet.
Step 2: Use conditional logic to check if it's a vowel.
A character is a vowel if it is 'a', 'e', 'i', 'o', 'u' (or their uppercase equivalents).
Step 3: Write the C code.
C

```
#include <stdio.h>

int main() {
    char ch;
    printf("Enter an alphabet: ");
    scanf(" %c", &ch);
    if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u' ||
        ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O' || ch == 'U') {
        printf("%c is a vowel.\n", ch);
    } else {
        printf("%c is a consonant.\n", ch);
    }
    return 0;
}
```

5. Display day name from number
Step 1: Read a day number from the user.
The program will take an integer from 1 to 7 as input.
Step 2: Use a switch statement to display the day name.
The switch statement is an efficient way to handle multiple possible values for a single variable.
Step 3: Write the C code.
C

```
#include <stdio.h>

int main() {
    int dayNum;
    printf("Enter a day number (1-7): ");
    scanf("%d", &dayNum);
    switch (dayNum) {
        case 1:
            printf("Sunday\n");
            break;
        case 2:
            printf("Monday\n");
            break;
        case 3:
            printf("Tuesday\n");
            break;
        case 4:
            printf("Wednesday\n");
            break;
        case 5:
            printf("Thursday\n");
            break;
        case 6:
            printf("Friday\n");
            break;
        case 7:
            printf("Saturday\n");
            break;
        default:
            printf("Invalid day number.\n");
            break;
    }
    return 0;
}
```